Configuring host agents using agent configuration file

Last Updated: 2026-05-12

You can apply the following configurations to the Instana agent by using the agent configuration file (`*instanaAgentDir*/etc/instana/configurati`

For a detailed list of all the Instana configuration parameters, see Instana Helm chart.

For Instana deployments on Kubernetes, see Configuring the Kubernetes agent by using the configuration file.

Note: The format of the agent configuration file is YAML, which is sensitive to blank spaces. Therefore, make sure to not use unnecessary spaces in t

Creating multiple configuration files

To provide modularity in configurations, you can create multiple `*instanaAgentDir*/etc/instana/configuration-<suffix>.yaml` files. Replace

All `*instanaAgentDir*/etc/instana/configuration-<suffix>.yaml` files are merged with `*instanaAgentDir*/etc/instana/configurat` `configuration-cde.yaml` file comes after the `configuration-abc.yaml` file. Nested structures that are specified in different configuration files such

See the following sections to learn how to use the agent configuration file to configure the agent:

Integrating the host agent with secret managers

Instana doesn't support encrypting confidential data such as password directly in the agent configuration file.

Therefore, when you add sensitive information (such as credentials or other information) in the agent configuration file, the contents might look like the follc

```
com.instana.example:
  user: 'instana'
  password: <password>
```

However, you might *not* want to put the sensitive information in plain text in the agent configuration file. In such circumstances, you can use secret manager retrieve them into the agent configuration file as required.

HashiCorp Vault

Note: The Vault integration requires the agent bootstrap version `1.2.9` or later.

Instana agent uses HashiCorp Vault to securely obtain values for sensitive settings in the agent configuration file.

You must provide the Vault integration configuration parameters to the host agent by adding the following lines to the agent configuration file:

```
com.instana.configuration.integration.vault:
  connection_url: <vault_server_base_url> # The address (URL) of the Vault server instance(e.g. http://127.0.0.1:8200 or https://exapmle.com:8200
  prefix: <optional_prefix> # Optional prefix path required if kv_version 2 is used and the /data/ must be injected further down. The /data/ segm
  token: <vault_access_token> # Vault access token with assigned, at least, `read` access policy to relevant Vault paths, optional if other auth
  github: # Optional auth method if Vault access token is not provided, has higher priority than approle if present
    github_token: <github_token> # Personal Access Token, must provide at least read:org scope, must be present if github is used as an auth prov
    auth_mount: github # Optional mount path for GitHub Auth, defaults to github
  approle: # Optional auth method if Vault access token or github auth is not provided
    role_id: <roleId> # AppRole RoleId, must be present if approle is used as an auth provider
    secret_id: <secretId> # AppRole SecretId, must be present if approle is used as an auth provider
    auth_mount: approle # Optional mount path for AppRole Auth, defaults to approle
  path_to_pem_file: <path_to_X.509_CA_certificate> # X.509 CA certificate (UTF8 encoded) in unencrypted PEM format, used by the Agent when commun
  secret_refresh_rate: 24 # This configuration option allows you to account for rotating credentials, refresh rate in hours, default 24
  kv_version: 2 # The Key/Value secrets engine version, default is 2
                # IMPORTANT: For KV version 2, the /data/ element is injected after the number of path elements in the prefix.
                #
                # Example configuration:
                #   prefix: foo/bar/blah  # 3 path elements
                #   secret_key:
                #     path: my/secret/vault/password
                #     key: password
                #
                # The agent will inject /data/ after 3 path elements (based on prefix depth): my/secret/vault/password → my/secret/vault/data/pas
```

For an example of configuring Vault with MySQL and adding secrets, see Sample Vault configuration for MySQL.

After the initial configuration of Vault, in the agent configuration file, you can specify the retrieval of secrets for various sensors.

See the following example in which the `password` is retrieved from HashiCorp Vault instead of directly adding it in the agent configuration file:

```
com.instana.example:
  user: 'instana'
  password:
    configuration_from:
      type: vault
      secret_key:
        path: <vault_path>
        key: <vault_secret_key>
```

You can replace the sensitive data string value with the YAML structure that specifies the Vault coordinates, and the host agent automatically retrieves the s

Also, you are not limited to integrating with secret managers for password fields. You can use the secrets managers for any configuration with a string as a v

For more information, see the following Vault concepts that are relevant for the Instana agent integration:

- Vault server
- Vault token management
- Access policy setup
- Vault KV secret engine
- Vault TCP listeners configuration

IBM Cloud Secrets Manager

The IBM Cloud Secrets Manager is based on open-source HashiCorp Vault and provides the same API and the same configuration as HashiCorp Vault. For m

Starting with version 1.0.11 of the Vault component, the IBM Cloud Secrets Manager SDK with IAM Keys are supported to be used in Vault.

You must provide the IBM Cloud Secrets Manager integration configuration parameters to the host agent by adding the following lines to the agent configura

```
com.instana.configuration.integration.vault:
  connection_url: <secrets-manager-address> # The address (URL) of the IBM Cloud Secrets Manager server instance(e.g. https://f022446e-1024-4aa9-
  ibm_secrets_manager: <iam_key> # IAM Key that can be used to create access tokens
  secret_refresh_rate: 24 # This configuration option allows you to account for rotating credentials, refresh rate in hours, default 24
```

You can find endpoint descriptions in IBM Cloud Docs - Secrets Manager or within the IBM Secrets Manager dashboard.

To create an IAM key, see IBM Cloud - IAM Keys.

After you integrate the IBM Cloud Secrets Manager to the host agent, in the agent configuration file, you can specify the retrieval of secrets for various sensc

See the following example in which the `password` is retrieved from the IBM Cloud Secrets Manager instead of directly adding it in the agent configuration fil

```
com.instana.example:
  user: 'instana'
  password:
    configuration_from:
      type: vault
      secret_key:
        path: <secret-id> # The id of the Secret within the IBM Secrets Manager, (e.g. cc32688d-89c0-6fa8-c0b4-6cc88c232e66)
        key: <kv-key-entry> # The Key inside the Secret Object of type KV (e.g. login)
      poll_rate: 300 # seconds
```

Sample Vault configuration for MySQL

See the following sample configuration in the `*instanaAgentDir*/etc/instana/configuration.yaml` file to integrate HashiCorp Vault with MySQL

> **Note:** Before you start configuring the Vault for MySQL, ensure that Vault and MySQL are installed and configured in your system.

```
com.instana.configuration.integration.vault:
  connection_url: http://127.0.0.1:8200
  prefix: secret
  token:  secret-vault-token
```

> **Note:** The `prefix` attribute is optional.

```
com.instana.plugin.mysql:
  user: 'myuser'
  password:
    configuration_from:
      type: vault
      secret_key:
        path: secret/my-secret
        key: password
```

> **Note:** The `/data/` path is not mentioned in the MySQL plug-in. If you use kv_version 2, omit any `/data/` within the path.

Run the following command to see the secrets:

```
vault kv get secret/my-secret
```

The secrets are displayed as follows:

```
==== Secret Path ====
secret/data/my-secret

======= Metadata =======
Key                Value
---                -----
created_time       2024-07-08T06:53:10.830770915Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1

====== Data ======
Key         Value
---         -----
password    *****
username    myuser
```

Kubernetes secrets

To retrieve sensitive information from Kubernetes secrets into the agent's sensor configuration, combine the following two concepts:

- Configure Kubernetes to mount secrets into environment variables or files:
  - This secret can be a workload-specific environment variable or file, which holds information only for the workload to be monitored, such as access crec
  - This secret can also be a general secret to be used in the agent configuration, which is mounted as an environment variable or file into the agent pod.
- Adapt the agent's `configuration.yaml` file to read configuration values from the mounted environment variables or files:
  - Read the secret from a workload-specific environment variable or file by using the `configuration_from: {type: env}` or `configuration_fror`
  - Read the secret from an agent environment variable or local file by using the `configuration_from: {type: agent_env}` or `configuration_fı`

By combining these two processes, your workloads can carry sensitive information in pod-specific or process-specific environment variables or files. In add information and apply the information to the workload-specific configuration.

For secrets which apply to the general agent configuration, you can retrieve the sensitive information from the agent environment.

Obtaining configuration from agent environment or agent local files

You can configure the agent to read configuration values from its own set of environment variables or local files. This approach is useful when you have ager your `configuration.yaml` file.

The agent reads the configuration values from its own set of environment variables (supports only YAML literals, such as a string, a Boolean, or a number), o maps).

An intentional symmetry exists between the capabilities of the host agent to read sensitive information from its local environment (variable or file) and the v supported Operating Systems and all supported container runtimes.

> **Note:** Numeric values in the environment variables and specific files are converted to integer values. The Boolean values `true` and `false` are conve

Obtaining configurations from agent environment

For configuration values that are taken from the agent's environment, modify the `configuration.yaml` file as follows:

```yaml
com.instana.configuration.integration.vault:
  connection_url: <vault_server_base_url>
  token:
    configuration_from:
      type: agent_env
      env_name: INSTANA_AGENT_VAULT_TOKEN
```

In the followiong example, the agent reads the access token for an HashiCorp Vault configuration from its own environment, so the token does not need to b

```
com.instana.configuration.integration.vault:
  connection_url: <vault_server_base_url>
  token:
    configuration_from:
      type: agent_env
      env_name: INSTANA_AGENT_VAULT_TOKEN
```

If you do not specify a default value, and if the environment variable is not present on the agent's environment, then the `configuration.yaml` file consid

Obtaining configurations from agent file system

For configuration values that are taken from a file, modify the `configuration.yaml` file as follows:

```
com.instana.plugin.mysql:
  user: 'instana'
  password:
    configuration_from:
      type: agent_file
      file_path: <absolute or relative path to a file containing the configuration value>
      default_value: <set this when the file is not found>
```

The property `file_path` can either hold the absolute path to a file or point to a file relative to the agent's install folder. On a Linux host, this location is typic `/opt/instana/agent/etc/vault_token`, the entry in `configuration.yaml` file can either be the absolute path `/opt/instana/agent/etc/vaul`

Obtaining configurations from process environment and files

You can configure the agent to read configuration values from the process that are to be monitored. This approach is useful in the following situations:

- When you do not want to hardcode any information in your `configuration.yaml`.
- When you have several instances of one technology, such as MySQL, on the same node, and each instance needs different configurations.

The agent reads the configuration values by looking up the values from specific environment variables (supports only YAML literals, such as a string, a Boole like lists and maps).

An intentional symmetry exists between these capabilities of the host agent and the way that you mount secrets in Kubernetes. However, these capabilities runtimes.

> **Note:** Numeric values in the environment variables and specific files are converted to integer values. The Boolean values `true` and `false` are conve

Obtaining configurations from process environment

For configuration values that are taken from the target process environment, modify the `configuration.yaml` file as follows:

```
com.instana.plugin.mysql:
  user: 'instana'
  password:
    configuration_from:
      type: env
      env_name: <environment variable name>
      default_value: <set this when no env var is found on this process>
```

If you do not specify a default value, and if the environment variable is not present on the process to be monitored, then the `configuration.yaml` file cor

Note: Notes: On Kubernetes, this function works with mounting ConfigMap entries as environment variables in your containers. Therefore, you need
containers as environment variables. Allow the configurations in the host agent to use those environment variables. The values from environment va
YAML structures, such as lists and maps, are not supported. You cannot change the plug-in `enabled/disabled` setting by using configurations from

Obtaining configurations from process file system

For configuration values that are taken from a file, modify the `configuration.yaml` file as follows:

```
com.instana.plugin.mysql:
  user: 'instana'
  password:
    configuration_from:
      type: file
      file_path: <absolute path to a file containing the configuration value>
      default_value: <set this when the file is not found>
      common_value: <set this in addition to the configuration value from file>
```

In the following example, the agent reads the `/jmx-config-map.yaml` file from the process file system with the property `type: file`. The `default_va`
later merged with every process-specific configuration and is common to all monitored processes:

```
com.instana.plugin.java:
  jmx:
    configuration_from:
      type: file
      file_path: /jmx-config-map.yaml
      default_value:
        - object_name: 'java.lang:type=OperatingSystem'
          metrics:
          - attributes: 'CommittedVirtualMemorySize'
            type: 'absolute'
      common_value:
        - object_name: 'java.lang:type=OperatingSystem'
          metrics:
          - attributes: 'ProcessCpuLoad'
            type: 'delta'
```

In the following example, the `/jmx-config-map.yaml` file is mounted from a Kubernetes ConfigMap into the application container that provides the follov

```
- object_name: 'java.lang:type=Compilation'
  metrics:
  - attributes: 'TotalCompilationTime'
    type: 'delta'
- object_name: 'java.lang:type=ClassLoading'
  metrics:
  - attributes: 'LoadedClassCount'
    type: 'absolute'
```

This process results in the following final configuration. Note the `common_value` addition for `ProcessCpuLoad`:

```
com.instana.plugin.java:
  jmx:
  - object_name: 'java.lang:type=Compilation'
    metrics:
    - attributes: 'TotalCompilationTime'
      type: 'delta'
  - object_name: 'java.lang:type=ClassLoading'
    metrics:
    - attributes: 'LoadedClassCount'
      type: 'absolute'
  - object_name: 'java.lang:type=OperatingSystem'
    metrics:
    - attributes: 'ProcessCpuLoad'
      type: 'delta'
```

Important

If the process that is to be monitored is running inside a container, then the absolute path of the file is resolved against that container's file system.

Note: Notes: Referencing files on the host's operating system when you try to resolve a configuration value for a containerized process is not suppor volumes in your containers. Therefore, you can define the configurations in ConfigMaps as follows: Mount the ConfigMaps to pods as files. Mount the configurations in the host agent to read particular configurations from the file path that you used to mount the ConfigMap volume inside the applicati configurations. The plug-in enabled/disabled cannot be changed by using the configurations from the process environment and specific files.

Monitoring additional file systems

By default, the agent monitors local file systems only. To add additional file systems, add the name of the file system (the first column in mtab or df).

For example, to monitor the `server:/usr/local/pub /pub nfs rsize=8192,wsize=8192,timeo=14,intr` file system, add the following line in th

```
com.instana.plugin.host:
  filesystems:
    - 'server:/usr/local/pub'
```

Specifying host tags

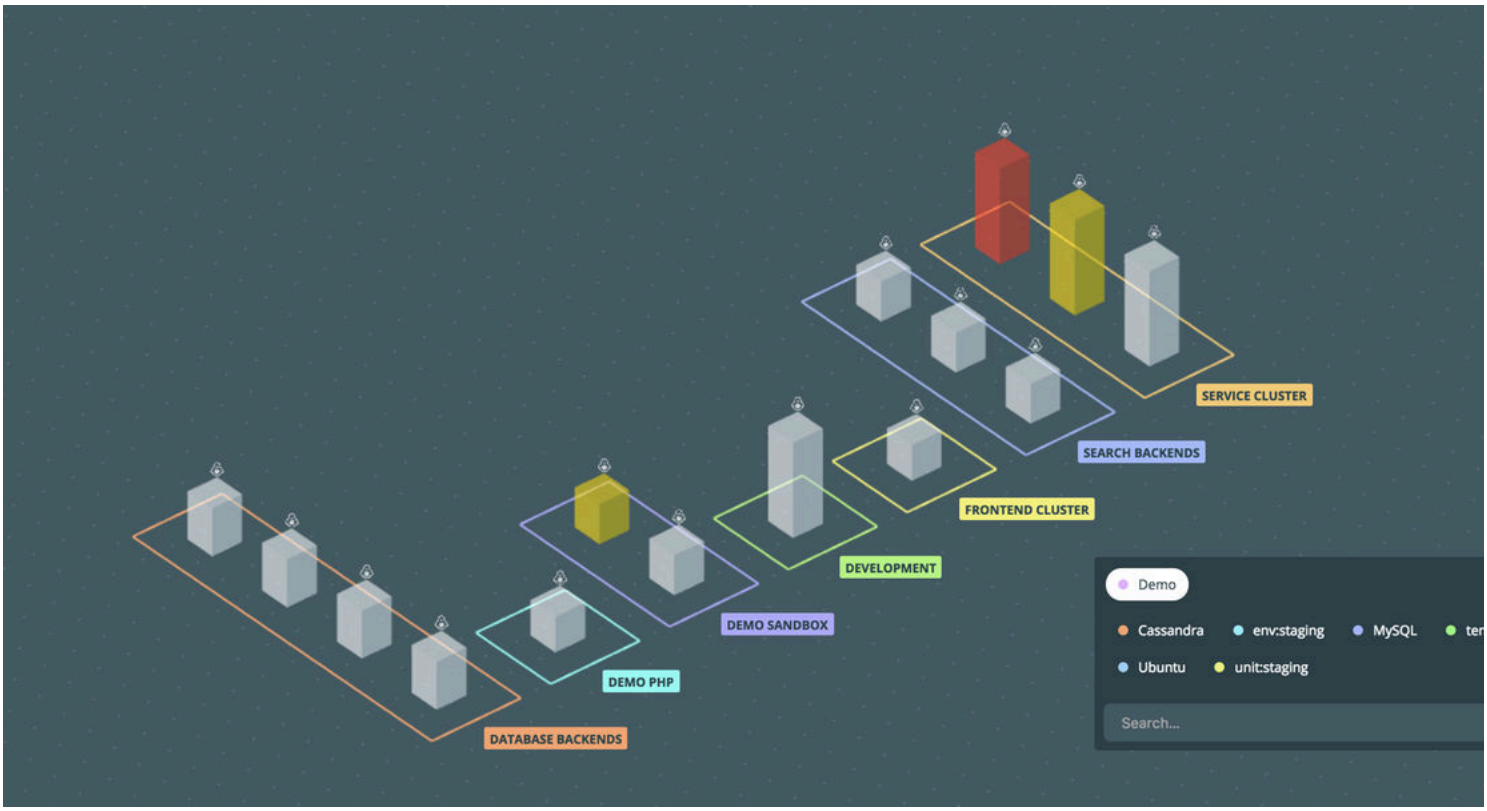
To specify tags for an agent in the `com.instana.plugin.host` configuration file, add a section in the following format:

```
com.instana.plugin.host:
  tags:
    - 'production'
    - 'app1'
```

Alternatively, you can use the `INSTANA_TAGS=production,app1` environment variable to specify tags for an agent (tags from environment variable and c

The `production` and `app1` tags are added to the specific agent, which enables searching and filtering of tags in the UI:

Figure 1. Infrastructure tags



Extracting the installed packages list

The host agent can report to Instana about the packages that are installed on the underpinning Linux operating system.

> **Note:** This configuration is disabled by default.

The following Linux distributions are supported:

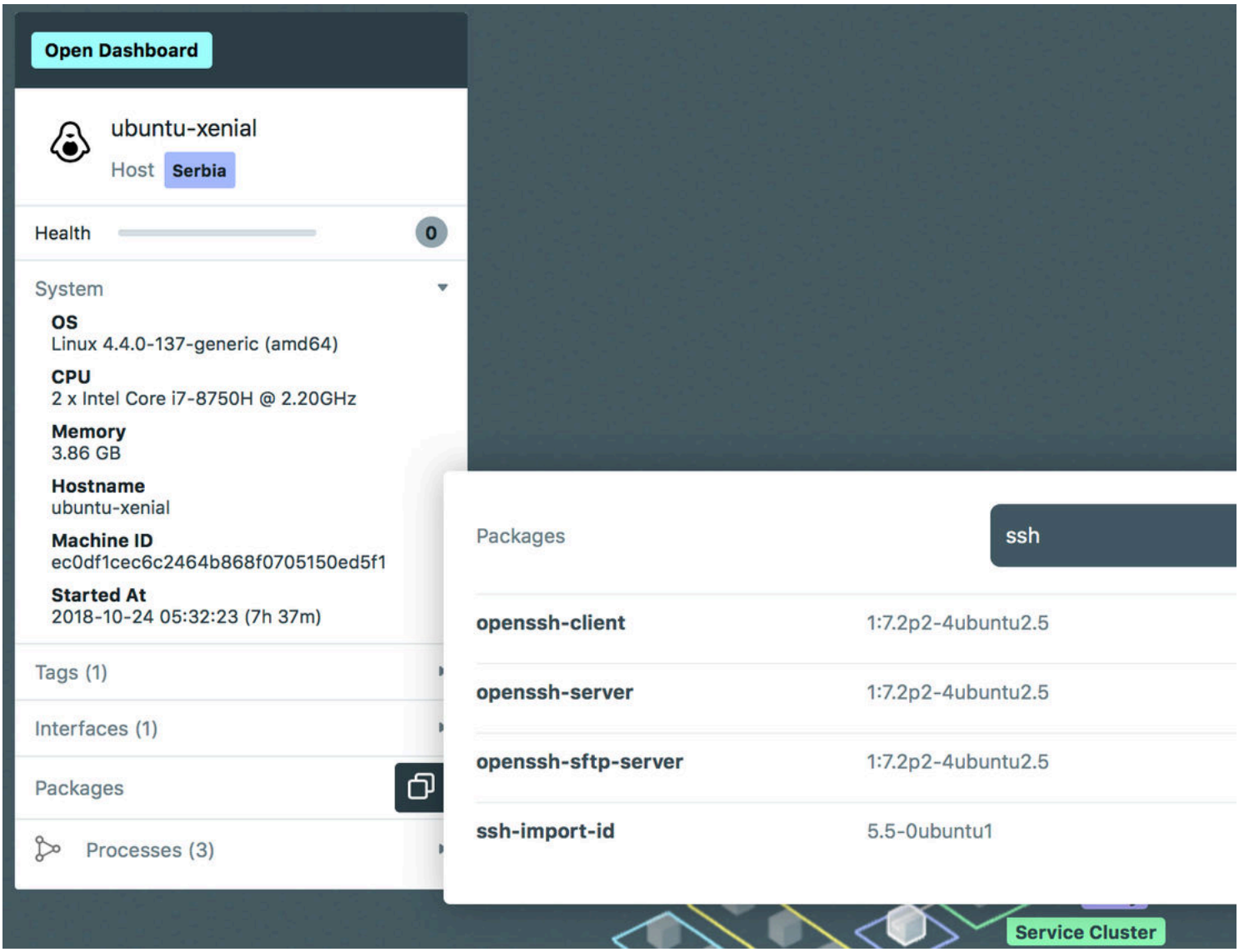
- Debian and its derivatives that use `dpkg` as package manager.
- Red Hat OpenShift and its derivatives that use `rpm` and `yum` as package managers.

To enable this feature, set the `collectInstalledSoftware` property to `true`.

```
com.instana.plugin.host:
  collectInstalledSoftware: false #  Valid values: true, false
```

After you enable the feature, the installed packages on an operating system are extracted once daily.

Figure 2. Package list



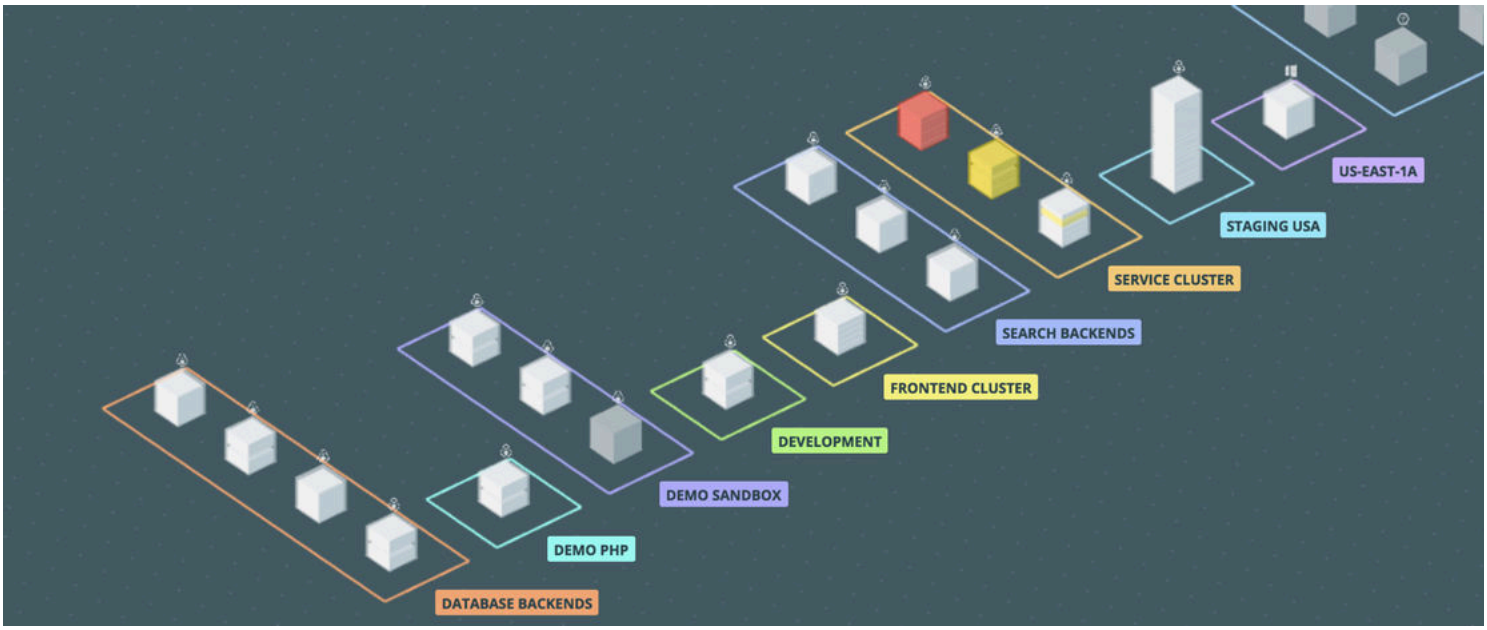
Setting custom zones

By default, Instana uses Amazon Web Services, Google Compute Engine, or OpenStack Nova availability zone information to group hosts. To customize the c
the section `com.instana.plugin.generic.hardware` by using the following format:

```
com.instana.plugin.generic.hardware:
  enabled: true
  availability-zone: 'Demozone'
```

You can also provide this configuration by using the `INSTANA_ZONE` environment variable, which then overrides the zone that is specified in the configuratic
map.

Figure 3. Infrastructure zones



Monitoring custom processes

By default, Instana automatically monitors the process metrics of higher-level sensors, such as Java® or MySQL. To monitor an OS process that is not autom
arguments:

```
com.instana.plugin.process:
  processes:
    - 'sshd'
    - 'slapd'
  arguments:
    - '/opt/script.sh'
```

Configuring secrets

Tracing data might contain sensitive data. Therefore, the Instana agent supports the specification of patterns for secrets (data to be redacted agent-side fro
processing, and therefore they are not available for analysis in the UI or retrieval by using API.

Specify the secrets as follows:

```
com.instana.secrets:
  # One of: 'equals-ignore-case', 'equals', 'contains-ignore-case', 'contains', 'regex'
  matcher: 'contains-ignore-case'
  list:
    - 'key'
    - 'password'
    - 'secret'
```

If you do not specify any custom secrets configuration, then Instana uses this secrets configuration by default. If a key matches an entry from the list, the va

Note: Generally, secrets are filtered from static data. When a process is running, filtering is highly optimized, and the changes that are made to confi
immediately, either restart the host agent or the running monitored component.

Instana supports secrets at the infrastructure and platform levels through the following configuration elements:

- Process environment variables
- Docker container information
- Kubernetes annotations and container environment variables Supporting secrets through this option requires an extra configuration. For more informatior

Instana supports secrets in the following runtime elements:

- Database connection strings
- HTTP query parameters (virtually for all supported runtimes, see the following support matrix), such as `https://my.domain/accounts/status?acc`
- HTTP matrix path parameters, such as `https://my.domain/accounts/account=*secret_1*;user=*secret_1*/status`.

The following runtimes are supported:

Language	Secrets in HTTP Query parameters	Secrets
Go	✓	×
Java®	✓	✓
NGINX	✓	×
Node.js	✓	×
.NET Core	✓	✓
.NET Framework	✓	✓
PHP	✓	✓
Python	✓	×
Ruby	✓	×
HTTPd	✓	×

Note: Notes: Instana does not support treating the arbitrary segments in a URL path as secrets. For example, https://my.domain/accounts/*secret*/ them require regular expressions to filter out the secrets, which is performance-intensive and therefore not encouraged. However, URLs such as http supported. Instana does not capture the following information that is related to the interaction with the databases: SQL parameters from stored proc literals from SQL queries and instead use parametrized queries. The Instana PHP sensor can strip SQL literals by using the sanitizeSql configuration

Capturing custom HTTP headers

By default, Instana doesn't collect HTTP headers when it traces HTTP calls.

If required, you can enable this feature by applying the following configuration in the agent configuration file:

```
com.instana.tracing:
  extra-http-headers:
    - 'x-request-id'
    - 'x-loadtest-id'
    - ...
```

The values are case-insensitive. The headers that are collected are shown in the call details (in the **Call details** section). You can also use the header names service configuration.

This feature currently has the following restrictions:

- All tracers capture *request* headers on HTTP *entries* (HTTP calls that the instrumented process receives).
- Some tracers might capture response headers on HTTP entries. See Table 2.
- Some tracers might capture request and response headers on HTTP exits (HTTP calls where the instrumented process is the client). See Table 2.
- If the same header is present as a request header and a response header, one of the two values might not be captured by the tracer.

Tracer	Request Headers on HTTP Entries	Response Headers on HTTP Entries	Request Headers on
Go	✅	✅	✅
Java	✅	✅ [1]	×
.NET	✅	×	×
Node.js	✅	✅	✅
PHP	✅	×	×
NGINX	✅	×	N.A.
Python	✅	✅	✅
Ruby	✅	✅	✅
HTTPd	✅	×	N.A.

Configuring Kafka trace correlation headers

You can configure the format for Kafka trace correlation headers that are used by Instana tracers by using the setting `com.instana.tracing.kafka.hea` example:

```
com.instana.tracing:
  kafka:
    header-format: string # possible values: binary, both, string
```

You must not disable the Kafka trace correlation entirely. However, if you need to disable the Kafka trace correlation entirely, then set `com.instana.trac`

```
com.instana.tracing:
  kafka:
    trace-correlation: false
```

Note: Many Kafka connectors use `SimpleHeaderConverter` as the default mechanism for handling Kafka message headers. This converter can fa values into native types, which might trigger numeric overflow errors. To resolve this issue, configure the Kafka Connect to use `StringConverter`:

```
header.converter=org.apache.kafka.connect.storage.StringConverter
```

For more information, see Kafka Header Migration.

Disabling tracing

You can disable tracing for specific span type (frameworks, libraries, or instrumentations) or entire groups of libraries (span category). For example, to exclu libraries, use the `disable` configuration option.

Note: Note: This feature is not yet supported by all tracers.

To configure this setting, define the types or categories that you want to disable under the `com.instana.tracing.disable` section in your agent configu

```
com.instana.tracing:
  disable:
```

```
    redis: true          # Disable Redis
    console: false       # Keep console enabled
    logging: true        # Disable the entire logging category
```

where:

- `true`: Disables tracing for the specified type or category.
- `false`: Explicitly keeps the specified type or category enabled.

In the preceding example, the configuration disables all instrumentations within the `logging` category, except for `console`, which is explicitly enabled. Ad spans are collected or reported for any disabled libraries or categories.

Support information

The following table lists the tracers that support disabling traces:

Tracer	Supports Disabling Traces
Node.js	✅
Go	✕
Java	✕
Python	✅
Ruby	✕
PHP	✅
.Net	✕
NGINX	✕

Capturing stack traces

You can configure stack trace capturing for exit spans across all your services. By default, tracers capture the last 10 call sites for every captured exit span.

> **Note:**
> - This feature is not yet supported by all tracers.
> - Stack traces of HTTP entry spans are typically not collected as they show only framework or runtime core code.

You can configure two aspects of stack trace capturing:

- **Stack trace length**: The number of call sites to be captured.
  - Supported values: 0–500
  - Default value: 10
- **Stack trace mode**: How stack traces are captured.
  - Supported values:
    - `all`: Collects stack trace for all exit spans (default).
    - `error`: Collects stack trace only for erroneous spans.
    - `none`: Doesn't collect stack trace.

To configure stack trace capturing, define the parameters under the `com.instana.tracing.global` section in your agent configuration file, as shown in t

```
com.instana.tracing:
  global:
    stack-trace-length: 15
    stack-trace: 'error'
```

In the preceding example, the configuration sets the stack trace depth to 10 call sites for all exit spans and uses the `all` mode.

Support information

The following table lists the tracers that support configuring stack trace capturing through agent configuration:

Table 1. Tracers that support stack trace configuration

Tracer	Supports stack trace configuration
Node.js	✅
Go	✕
Java	✕
Python	✅
Ruby	✅
PHP	✕
.Net	✕
NGINX	✕

Ignoring endpoints

You can exclude specific endpoints from tracing. For example, if you are using the `redis` package and want to avoid the tracing of commands, such as GET,

Note: Notes: The ignore endpoints feature currently has the following restrictions: Only specific packages are supported. Not all tracers are supporte

For more information about the specific packages and tracers that support this feature, see Support information.

Filtering options

You can filter the traces by using the following options:

- Filtering by method name: With this option, you can filter traces based only on method names. It is useful when you want to ignore specific operations, su
- Filtering by method name and endpoint: With this option, you can exclude traces based on both method and specific endpoints. This option is especially u method (for example, `consume`) but only for certain topics (for example, `topic1` or `topic2`).

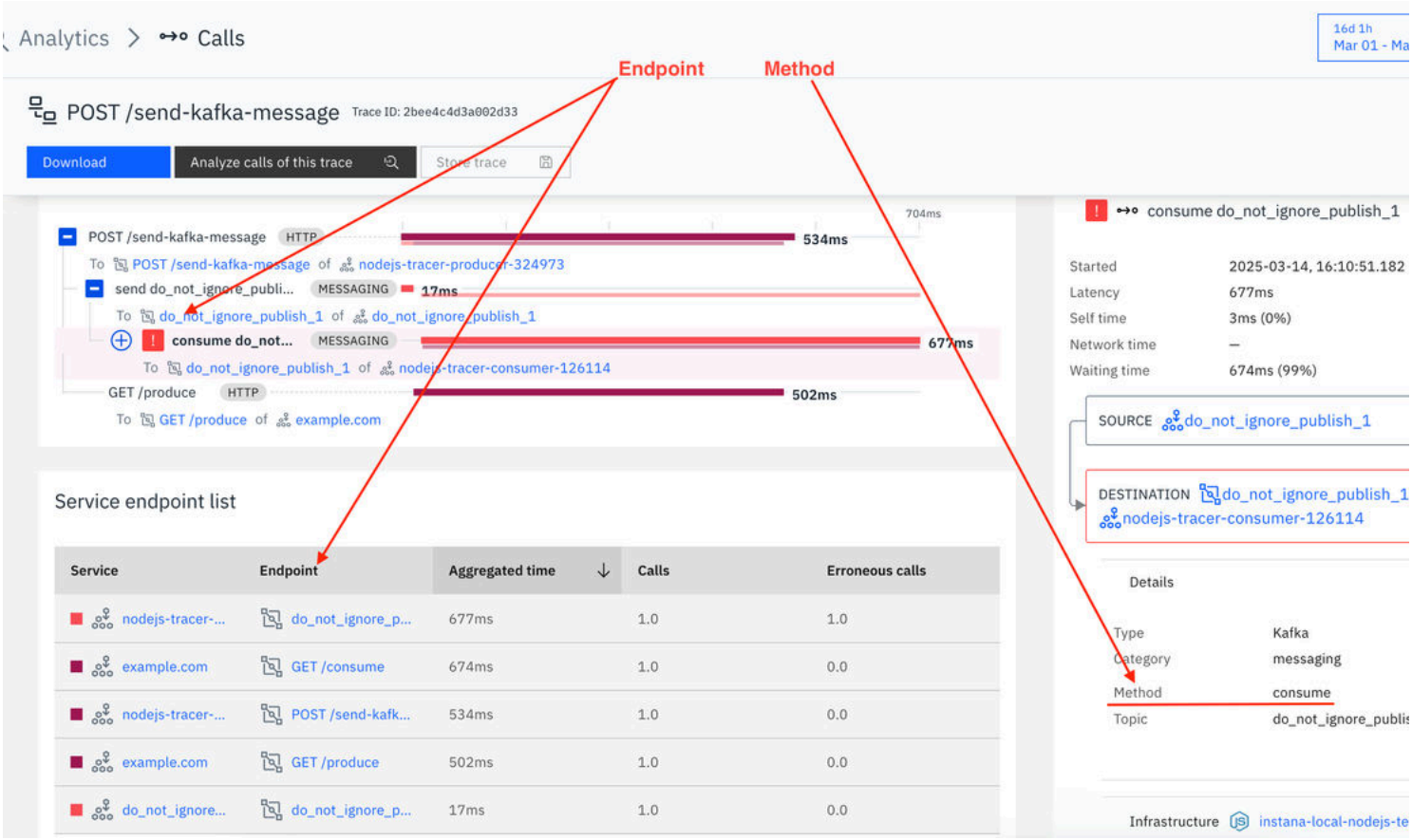
Filtering rules

The rules for filtering traces are as follows:

- When a trace is ignored, all subsequent downstream traces are also ignored.
- Use `*` to ignore all endpoints or methods.
- Endpoint values (such as Kafka topic names) remain consistent across services.

– Method names might vary depending on the programming language and technology. To determine the correct method and endpoint for your service, refer

The following Instana UI screenshot provides a visual reference for identifying the correct method and endpoint for configuration:



Configuring endpoints to be excluded

Note: When your system uses multiple services in different programming languages, make sure that all necessary method names are included in the

To configure the endpoints to be ignored, specify the endpoints that must be excluded from monitoring in the `com.instana.tracing.ignore-endpoin`

```yaml
com.instana.tracing:
  ignore-endpoints:

    # Filtering by Method Name
    redis:
      - 'get'
      - 'type'
    dynamodb:
      - 'query'
    kafka:
      - 'send'

    # Filtering by Method Name and Endpoint for Kafka
    kafka:
      - methods: ["consume"]
        endpoints: ["topic1", "topic2"]  # Exclude consume calls for topic1 and topic2

      - methods: ["consume", "send"]
        endpoints: ["topic3"]  # Exclude both consume and send calls for topic3

      - methods: ["*"]
        endpoints: ["topic4"]  # Exclude all methods for topic4

      - methods: ["consume"]
        endpoints: ["*"]  # Exclude consume method for all topics
```

In the preceding example, the following traces are ignored for the listed filtering options:

- Filtering by method (`Redis`, `DynamoDB`, and `Kafka`)
  - `GET` and `TYPE` commands in `Redis`
  - `QUERY` command in `DynamoDB`
  - `SEND` method in `Kafka` and all downstream traces
- Filtering by method and endpoint (`Kafka` only)
  - `CONSUME` method for `topic1` and `topic2` in `Kafka` and all downstream traces
  - `CONSUME` and `SEND` methods for `topic3` in `Kafka` all downstream traces
  - All methods (`*`) for `topic4` in `Kafka` and all downstream traces
  - `CONSUME` method for all topics (`*`) and all downstream traces

Support information

The following table lists the tracers and packages that support ignoring endpoints:

Supported packages	Node.js	Java	Go	PHP	Python
Redis	✓	✓	×	×	✓
DynamoDB	✓	✓	×	×	✓
Kafka	✓	✓	×	×	✓
HTTP	✓	×	×	×	×

Note: For Node.js, HTTP filtering applies to only incoming (entry) requests. Outgoing (exit) HTTP calls cannot be filtered.

Ignoring processes

You can exclude specific processes from tracing.

To ignore monitoring of certain processes, uncomment the `com.instana.ignore` section in the agent configuration file and list all the process names of th

You can define either processes or arguments. For example, if you define a process called `nginx` and an argument called `/opt/server/server.js`, both `/opt/server/server.js`) are ignored. See the following example:

```
com.instana.ignore:
  processes:
    - 'java'
    - 'httpd'
  arguments:
    - '-batch-file=/tmp/batch.def'
```

Important

For Windows, the process names in the list are case-sensitive.

Note: Processes and arguments are not related or codependent.

Sometimes, scripts or other jobs are started through the same command, but only the execution of a single command must be ignored. In this scenario, use

When you specify arguments, make sure to retrieve arguments and environmental variables that are defined in **Process** > **Arguments** in the Instana UI.

Instana takes in arguments based on the blank spaces between arguments. For example, if you run a command with `/usr/bin/java -p 4654 -Djava.` Instana to identify them as two different arguments.

Deactivating agent capabilities that are enabled in Instana UI

If you want to deactivate all the agent capabilities that are enabled in the Instana UI, add the following line to the `*instanaAgentDir*/etc/instana/c`

```
backchannel.enabled = false
```

Uploading code source files to Instana

The Instana agent can retrieve on-demand source code of the processes that it monitors and associate it with the collected tracing data.

To disable the on-demand upload of the source files into Instana, complete the following steps:

1. Set the following configuration in the `*instanaAgentDir*/etc/instana/com.instana.agent.main.config.Agent.cfg` configuration file:
   ```
   source.download.enabled = false
   ```
2. Restart the Instana agent to apply this setting.

Configuring error report events (only AIX operating system)

To specify the time interval to poll the error events from AIX system, set the `aixEventsPollRate` event in seconds (minimum is 900 seconds).

```
com.instana.plugin.host:
  aixEventsPollRate: 900 # (Optional) Only for AIX systems. Setting a value to 900 seconds or larger will enable error report event collection an
```

To disable the feature, remove `aixEventsPollRate` event from the agent configuration files.

Monitoring specific processes

Instana automatically monitors processes with `gunicorn`, `uwsgi`, or `python` names in their command line. You can include a specific process name in the name in the command line.

To add the process that you want to discover into the `configuration.yaml` file, use the following configuration:

```
com.instana.plugin.python:
  include_processes:
    - ./process_name
```

This configuration adds the process name to the monitored processes list, and you can view it in the Instana UI.

1. Supported technologies include `Servlet`, `Spring`, `Tomcat`, and `http4s`. ↩